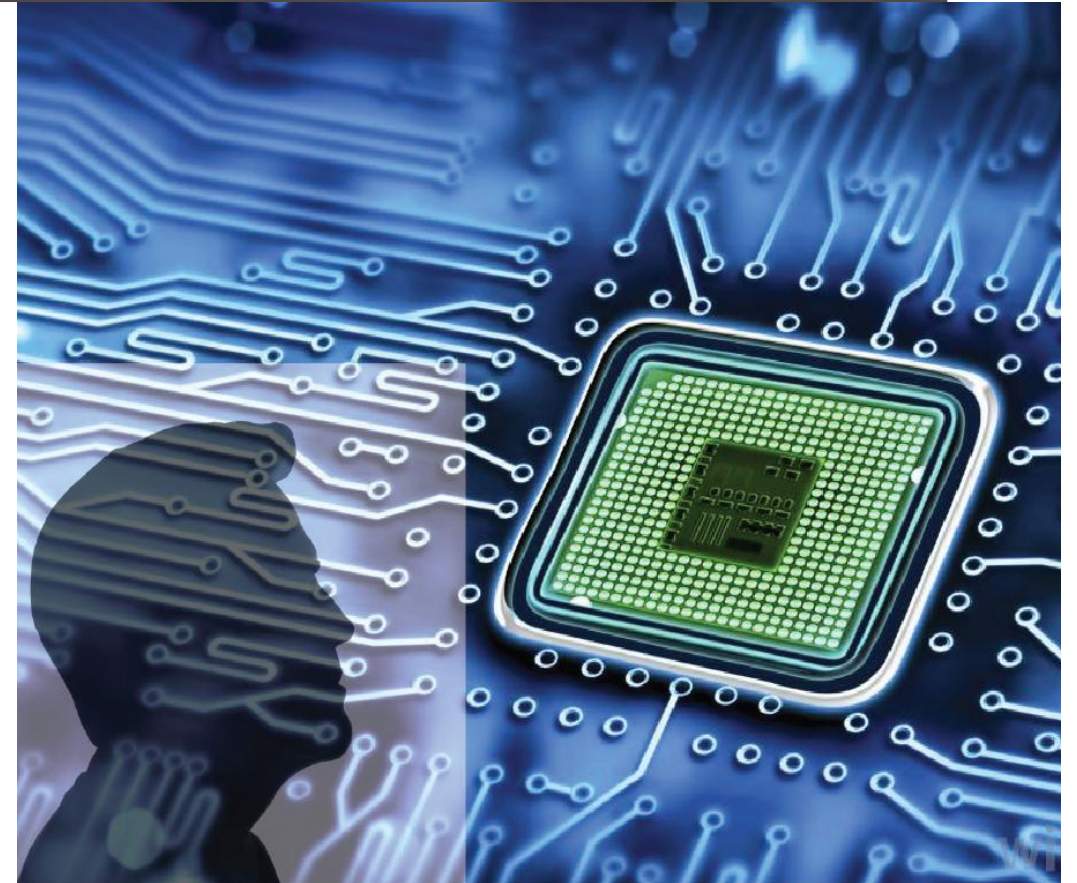


ADVANCED DIGITAL DESIGN AND VERIFICATION

Week-8 Lecture-1 SystemVerilog

Sneh Saurabh
7th October, 2025



Associative Arrays: Corrections and Clarification (1)

- Can use **foreach** to iterate
 - This is incorrect
- LRM says: Associative arrays that specify a wildcard index type **shall not be used in a foreach loop**

```
initial begin
    logic [63:0] assoc[*], idx = 1;

    // Initialize widely scattered values
    repeat (64) begin
        assoc[idx] = idx;
        idx = idx << 1;
    end

    // Step through all index values with foreach
    foreach (assoc[i])
        $display("assoc[%h] = %h", i, assoc[i]);

    // Step through all index values with functions
    if (assoc.first(idx))
        begin // Get first index
            do
                $display("assoc[%h]=%h", idx, assoc[idx]);
            while (assoc.next(idx)); // Get next index
        end

    // Find and delete the first element
    assoc.first(idx);
    assoc.delete(idx);

end
```

Associative Arrays: Corrections and Clarification (2)

Order of Data

- LRM says:
 - Associative arrays that specify a wildcard index type have the following properties:
 - The ordering is numerical (smallest to largest).

```
ncsim> run
assoc[000000000000000002]=000000000000000004
assoc[000000000000000003]=000000000000000006
assoc[000000000000000005]=00000000000000000a
assoc[000000000000000007]=00000000000000000e
assoc[000000000000000009]=000000000000000012
```

```
module top();

initial begin

    logic [63:0] assoc[*], idx = 1;

    assoc[5]=5*2;
    assoc[2]=2*2;
    assoc[9]=9*2;
    assoc[3]=3*2;
    assoc[7]=7*2;

    // Step through all index values with functions
    if (assoc.first(idx))
    begin // Get first index
        do
            $display("assoc[%h]=%h", idx, assoc[idx]);
        while (assoc.next(idx)); // Get next index
    end

end

endmodule
```

Behavior always_comb, always_latch (genus)

```
module top(a, b, s, out1);  
  input a, b, s;  
  output out1;  
  logic out1;
```

```
  always_latch begin  
    out1 = a & b & s;  
  end
```

```
endmodule
```

```
module top(a, b, s, out1);  
  input a, b, s;  
  output out1;  
  logic out1;
```

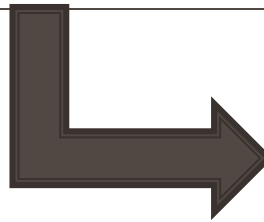
```
  always_comb begin  
    if (s) out1 = a & b;  
  end
```

```
endmodule
```

CDFG-368	Warning	1	Generated logic differs from the expected logic. The logic generated for an always_comb, always_latch or always_ff process may not match the specified behavior.
----------	---------	---	---

CDFG2G-615	Warning	1	Generated logic differs from the expected logic. The logic generated for an always_comb, always_latch or always_ff process may not match the behavior specified in the input HDL.
CDFG2G-616	Info	1	Latch inferred. Check and revisit your RTL if this is not the intended behavior.

SystemVerilog



Procedural Blocks and
Routines

Procedural Blocks: Enhancements (1)

- `break` and `continue` in loop

```
module top();  
  initial begin  
    int i;  
    i = 0;  
    do begin  
      if (i == 4) break;  
      i++;  
      if (i < 2) continue;  
      $display("i=%0d", i);  
    end while (1);  
  end  
endmodule
```

```
ncsim> run  
i=2  
i=3  
i=4
```

Procedural Blocks: Enhancements (2)

- Function is allowed to not return value
- C-style declaration of **task** and **function** arguments

```
module top();  
  logic [3:0] A, B, S;
```

```
  function void myFunc();  
    $display("Void Function A=%d B=%d S=%d", A, B, S);  
  endfunction
```

```
  task myTask(logic [3:0] x, y, z);  
    $display("Task C-style A=%d S=%d S+2=%d", x, y, z);  
  endtask
```

```
  initial begin  
    A = 5; B=7; S = A+B;  
    myFunc();  
    myTask(A, S, S+2);  
  end
```

```
endmodule
```

```
Void Function A= 5 B= 7 S=12  
Task C-style A= 5 S=12 S+2=14
```

Procedural Blocks: Enhancements (3)

- Allows default types (automatically derived type)
- If direction and type not specified then default is assumed
 - Either **input logic**
 - Or what was specified in the previous argument

```
module top();
```

```
task myTask1(a, b, s); // default input logic
    $display("Task 1 A=%d B=%d S=%d", a, b, s);
endtask
```

```
task myTask2(logic [3:0] a, b, s); // default logic [3:0]
    $display("Task 2 A=%d B=%d S=%d", a, b, s);
endtask
```

```
initial begin
    myTask1(5, 7, 5+7);
    myTask2(5, 11, 5+11);
end
endmodule
```

```
Task 1 A=1 B=1 S=0
Task 2 A= 5 B=11 S= 0
```


Procedural Blocks: Enhancements (4)

- Allows specifying a default value to an argument
 - Default value is used if its value is left out in the call to that function
 - Helpful when more arguments are added to a function later on
 - Can also call argument by name (similar to a port)

```
module top();  
  
    task myDefaultArguments(int a=1, b=2, c=3);  
        $display("Arguments A=%0d B=%0d C=%0d", a, b, c);  
    endtask  
  
    initial begin  
        myDefaultArguments();  
        myDefaultArguments(5);  
        myDefaultArguments(7, ,9);  
        myDefaultArguments(.a(7), .c(9));  
        myDefaultArguments(7,8,9);  
    end  
endmodule
```

```
ncsim> run  
Arguments A=1 B=2 C=3  
Arguments A=5 B=2 C=3  
Arguments A=7 B=2 C=9  
Arguments A=7 B=2 C=9  
Arguments A=7 B=8 C=9
```

Procedural Blocks: Enhancements (5)

- Enable arguments passing by reference rather than value
 - Passing by value leads to copying onto the stack which can be very expensive for big arrays

```
ncsim> run
Array[99]=25
Array[100]=25
Array[99]=25 Array[100]=35
```

- By default: **task** defined in a module is **static**
 - Can override using keyword **automatic**
- For static task/function, it is illegal to pass by reference

```
module top;
  int bigArray[];
```

```
  task myPassValue(int arr[]);
    $display("Array[99]=%0d", arr[99]);
    arr[99] = 30;
  endtask
```

```
  task automatic myPassReference(ref int arr[]);
    $display("Array[100]=%0d", arr[100]);
    arr[100] = 35;
  endtask
```

```
  initial begin
    bigArray = new[100000];
    bigArray[99] = 25;
    bigArray[100] = 25;
    myPassValue(bigArray);
    myPassReference(bigArray);
    $display("Array[99]=%0d Array[100]=%0d",
      bigArray[99], bigArray[100]);
  end
endmodule
```

Procedural Blocks: Enhancements (6)

- System Verilog also allows using the **const** modifier which allows compiler to check whether an argument is modified
- Use **const ref** wherever applicable

```
module top;

    int bigArray[];

    task automatic myPassConstReference(const ref int arr[]);
        $display("Array[100]=%0d", arr[100]);
        arr[100] = 35;
    endtask

    initial begin
        bigArray = new[100000];
        bigArray[100] = 25;
        myPassConstReference(bigArray);
        $display("Array[100]=%0d", bigArray[100]);
    end
endmodule
```

```
file: pass_const_reference.sv
    arr[100] = 35;
```

```
ncvlog: *E,CONASN (pass_const_reference.sv,7|4): Constant variable cannot be
assigned outside of an initialization.
```

SystemVerilog



Classes

SystemVerilog: Classes

- SystemVerilog supports classes to encapsulate data and functions.
- The data are referred to as class properties and functions are called methods.
- Can define and use classes in SystemVerilog in a **program**, **module**, **package**, or outside of any of these.

- The instances of a class are referred to as objects.
- We can create an object of a class by declaring a variable of that class type and then assigning it to the result of the function **new**.
- We can then call various methods and access class properties on that object.

```
class Bus;  
  
    bit [31:0]addr;  
    function void display;  
        $display("Bus: %h", addr);  
    endfunction : display  
  
endclass : Bus
```

```
module top;  
  
    initial  
    begin  
        Bus b;  
        b = new();  
        b.display();  
    end  
  
endmodule
```

Classes: Role of Constructor

The role of **new** function

- Before **new()** is called the handle to object (b) is initialized to null
- **new()** allocates space for object (similar to malloc in C)
 - Size of object, plus some memory for house-keeping (such as objects type)
- It initializes the variables to their default values (0 for 2-state and X for 4-state)
- Returns the address of the new object

Bus: 00000000

```
class Bus;  
  
    bit [31:0]addr;  
    function void display;  
        $display("Bus: %h", addr);  
    endfunction : display  
  
endclass : Bus  
  
module top;  
  
    initial  
    begin  
        Bus b;  
        b = new();  
        b.display();  
    end  
  
endmodule
```

Classes: Customized Constructor

Customized constructor can be used to

- Initialize the variables to customized values
- Can use default values or can be passed during instantiation

```
class BusTran;  
  logic [31:0] addr, crc, data[8];  
  function new(input logic [31:0] a=3, d=5);  
    addr = a;  
    data = '{default:d};  
  endfunction
```

```
  function void display;  
    $display("addr: %h crc: %h", addr, crc);  
    foreach (data[i])  
      $display("data[%0d]: %h", i, data[i]);  
  endfunction : display
```

```
endclass
```

```
module top;  
  initial  
  begin  
    BusTran b;  
    b = new(.a(9));  
    b.display();  
  end  
endmodule
```

```
ncsim> run  
addr: 00000009 crc: xxxxxxxx  
data[0]: 00000005  
data[1]: 00000005  
data[2]: 00000005  
data[3]: 00000005  
data[4]: 00000005  
data[5]: 00000005  
data[6]: 00000005  
data[7]: 00000005
```

```
class BusTran;  
  logic [31:0] addr, crc, data[8];  
  function new;  
    addr = 3;  
    data = '{default:5};  
  endfunction
```

```
  function void display;  
    $display("addr: %h crc: %h", addr, crc);  
    foreach (data[i])  
      $display("data[%0d]: %h", i, data[i]);  
  endfunction : display  
endclass
```

```
module top;  
  initial  
  begin  
    BusTran b;  
    b = new();  
    b.display();  
  end  
endmodule
```

```
ncsim> run  
addr: 00000003 crc: xxxxxxxx  
data[0]: 00000005  
data[1]: 00000005  
data[2]: 00000005  
data[3]: 00000005  
data[4]: 00000005  
data[5]: 00000005  
data[6]: 00000005  
data[7]: 00000005
```

Which **new** function to call?

- SystemVerilog looks on the type of handle on the LHS of assignment and calls its new

Classes: Difference between Object and Handle to Object

- We declare a handle to an object and can use the handle multiple times with different objects
 - Creating objects dynamically
- We construct an object using `new` and use it subsequently, and it can be deallocated

```
class Bus;  
  bit [31:0] addr;  
  function new(input logic [31:0] a=3);  
    addr = a;  
  endfunction  
  function void display;  
    $display("Bus: %h", addr);  
  endfunction : display  
endclass : Bus
```

```
module top;  
  initial  
  begin  
    Bus b1, b2;  
    b1 = new(5);  
    b1.display();  
    b2 = b1;  
    b1.display(); b2.display();  
    b1 = new(7);  
    b1.display(); b2.display();  
  end  
endmodule
```

```
ncsim> run  
Bus: 00000005  
Bus: 00000005  
Bus: 00000005  
Bus: 00000007  
Bus: 00000005
```


Classes: Object Deallocation

- SystemVerilog perform Garbage Collection (in contrast to C/C++, similar to Java)

Garbage collection:

- Automatically frees objects (deallocates memory) that are no longer referenced
- Keeps track of number of handles that point to an object
- When this number reduces to 0, SystemVerilog releases the memory for it
- Implementation of garbage collection in simulators can vary, one easy way is **reference counting**

```
1 class Bus;
2   bit [31:0]addr;
3   function new(input logic [31:0] a=3);
4     addr = a;
5   endfunction
6   function void display;
7     $display("Bus: %h", addr);
8   endfunction : display
9 endclass : Bus
10
11 module top;
12   initial
13   begin
14     Bus b; // Declare and create a handle
15     b = new(5); // Allocate and construct an object,
16               // handle b references it
17
18     b.display();
19     b = new(6); // Allocate and construct a second object, first object
20               // can be freed because no handle references it
21     b.display();
22     b = null; // Second object can also be freed now, because no
23             // handle references it
24     b.display(); // error to use the object using handle
25               // because the handle references null now
26   end
27
28 endmodule
```

```
ncsim> run
Bus: 00000005
Bus: 00000006
Bus: ncsim: *E,TRNULID: NULL pointer dereference.
      File: ./class4.sv, line = 7, pos = 27
      Scope: worklib.$unit_0x405781d7::Bus::display
      Time: 0 FS + 0
Verilog Stack Trace:
0: function worklib.$unit_0x405781d7::Bus::display at ./class4.sv:7
1: initial block in top at ./class4.sv:24

./class4.sv:7      $display("Bus: %h", addr);
ncsim> exit
```

Classes: Limiting Access to Variables

Three access types

- Default access is public
 - Other code can read and write properties (variables) and call all methods (routines).
 - There is no keyword for this behavior.
- **protected**
 - Only the derived class can access
 - Need to give get/set functions if needed
- **local**
 - Most restrictive
 - Does not let anyone, not even the derived class, access this
 - Need to give get/set function if needed

```
class Base;
  int publicVal;
  protected int protectVal;
  local int privateVal;
  function void displayBase;
    $display("publicVal: %0d", publicVal);
    $display("protectVal: %0d", protectVal);
    $display("privateVal: %0d", privateVal);
  endfunction
endclass : Base

class Derived extends Base;
  function void displayDerived;
    $display("publicVal: %0d", publicVal);
    $display("protectVal: %0d", protectVal);
    // $display("privateVal: %0d", privateVal); // cannot access this
  endfunction
endclass

module top;
  initial
  begin
    Base b;
    b = new();
    b.publicVal = 8;
    // b.protectVal = 9; //Error
    // b.privateVal = 12; // Error
    b.displayBase();
  end
endmodule
```

- Class can be derived from a base class using **extends** keyword

```
publicVal: 8
protectVal: 0
privateVal: 0
```

Classes: Defining methods outside class

- Using **extern** keyword in class for declaration
- Use **::** in definition outside class

```
class Bus;  
    bit [31:0]addr;  
    extern function void display;  
endclass : Bus
```

```
function void Bus::display;  
    $display("Bus: %h", addr);  
endfunction : display
```

```
module top;  
    initial  
    begin  
        Bus b;  
        b = new();  
        b.display();  
    end  
endmodule
```

```
ncsim> run  
Bus: 00000000  
ncsim: *W,RNQUIE: Simulation is complete.  
ncsim> exit
```

Classes: Static Variables (1)

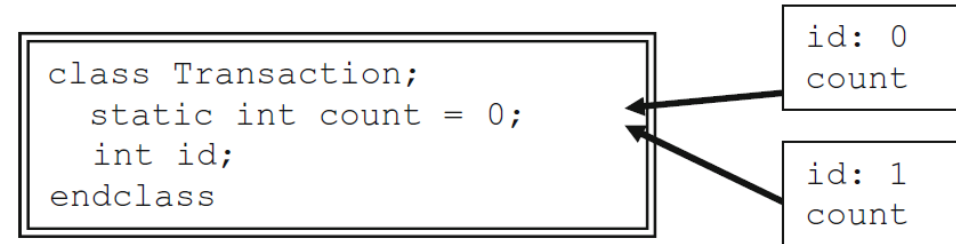
- Sometimes we need a variable that is shared by all objects of a certain type
 - We can create **static** variable inside the class
 - Shared among all objects of that class (only one copy of that variable for a class)

```
class Transaction;
  static int count = 0; // one copy for entire class
  int id; // unique to each object
  function new();
    id = count++;
  endfunction
  function void display;
    $display("Transaction: No. of objects %0d Id of object %0d", count, id);
  endfunction : display
endclass

module top;

  initial
  begin
    Transaction t1;
    Transaction t2;
    t1 = new();
    t2 = new();
    t1.display();
    t2.display();
  end

endmodule
```



```
Transaction: No. of objects 2 Id of object 0
Transaction: No. of objects 2 Id of object 1
```

Classes: Static Variables (2)

- Static variables can also be accessed without object

```
Transaction t1;  
t1 = new();  
$display("Transaction: No. of objects %0d  
Id of object %0d", Transaction::count, t1.id);
```

- A method can be defined as **static**
 - Can be called even when no object is created
 - Can read/write only static class properties (variables)

References

- Spear, Chris. SystemVerilog for Verification: A Guide to Learning the Testbench Language Features. Vol. 161. Springer, 2008.